

Open-Source Crypto on ESP32 (x25519, ed25519, HKDF, AES-GCM)

Executive Summary: ESP32's official SDK (ESP-IDF) includes a fork of MbedTLS (Apache-2.0/GPL-2.0+) [62†L1354-L1362] that supports a wide range of ciphers and TLS protocols. Modern primitives like X25519 (Curve25519 ECDH) and Ed25519 (EdDSA) are available in newer MbedTLS/PSA versions, but Ed25519 is *not yet* implemented in MbedTLS v4.x [65†L3478-L3482] [65†L3568-L3572]. Alternative libraries (e.g. WolfSSL, NoxTLS) fill these gaps: NoxTLS explicitly supports X25519/Ed25519, HKDF-SHA256, AES-GCM, etc. [22†L43-L51], and WolfSSL likewise supports Curve25519 and Ed25519 [66†L58-L62]. All ESP32 variants (ESP32, S2, S3, C3, H2, etc.) include hardware accelerators for AES and SHA2, and many support accelerated RSA/ECC (P-256) and MPI math. Enabling `CONFIG_MBEDTLS_HARDWARE_AES`, `HARDWARE_SHA`, `HARDWARE_MPI` in menuconfig offloads AES, SHA, and big-number ops to hardware [9†L294-L302]. In practice, benchmarks on an ESP32-S3 @240 MHz show AES-128-GCM ~1.35 MB/s and hardware SHA256 ~26 MB/s [40†L100-L108] [40†L121-L129]. Ed25519 signing/verifying takes ~26 ms/~24 ms (software) and X25519 key exchange ~14-15 ms [40†L169-L177] [40†L210-L214]. Memory/flash use depends on enabled algorithms: by default MbedTLS allocates 16 KB buffers (32 KB total TX/RX) for TLS fragments [47†L3287-L3294], so reducing `CONFIG_MBEDTLS_SSL_MAX_CONTENT_LEN` in menuconfig can save heap. The following sections detail toolchains, library versions, hardware use, examples, and security tips, with code snippets and build instructions.

ESP32 mbedTLS and Hardware Crypto

ESP-IDF and mbedTLS versions: Each ESP-IDF release tracks a mbedTLS release (forked by Espressif). For example, ESP-IDF v6.0 (current) uses mbedTLS v4.1.x [12†L212-L219]. Earlier IDF v5.x used mbedTLS v3.6.x. Espressif's fork adds patches for hardware offload (MPI big-number, ECC) [12†L202-L208].

Hardware Acceleration: All Xtensa/RISC-V ESP32 chips include hardware AES and SHA engines. Many also include RSA/ECDSA/DH offload (especially on S2/S3/C3/C6). In menuconfig under "Component config → mbedTLS", enable:

- `CONFIG_MBEDTLS_HARDWARE_AES=y` to use AES accelerator,
 - `CONFIG_MBEDTLS_HARDWARE_SHA=y` to use SHA256/224 accel,
 - `CONFIG_MBEDTLS_HARDWARE_MPI=y` to offload multi-precision math (RSA/ECDH) [9†L294-L302].
- On supported chips, this lets mbedTLS automatically use the crypto peripherals. (For example, using `esp_sha256` hardware yields ~26 MB/s vs ~2.3 MB/s software [40†L121-L129].) In NoxTLS or WolfSSL components, similar config switches enable ESP-IDF crypto HALs [61†L169-L177].

Toolchains and Configuration: ESP-IDF projects use `idf.py`. The default ESP-IDF build includes mbedTLS; no extra library fetch is needed. To use NoxTLS, add it as a component (e.g. `idf.py add-dependency "argenox/noxtls"` [61†L132-L140]) and set in `sdkconfig.defaults`:

```

CONFIG_NOXTLS=y
CONFIG_NOXTLS_PROFILE_MINIMAL_TLS_CLIENT=y
CONFIG_NOXTLS_USE_ESP_ENTROPY=y
CONFIG_NOXTLS_ESP_HW_ECC=y
CONFIG_NOXTLS_ESP_HW_ECDSA=y
CONFIG_NOXTLS_ESP_HW_MPI=y
CONFIG_NOXTLS_ESP_HW_AES=y
CONFIG_NOXTLS_ESP_HW_SHA=y
CONFIG_MBEDTLS_HARDWARE_MPI=y
CONFIG_MBEDTLS_HARDWARE_SHA=y
CONFIG_MBEDTLS_HARDWARE_AES=y

```

This enables NoxTLS and hardware offload flags [【61†L138-L147】](#) . WolfSSL can be used via Espressif's wolfSSL component and menuconfig (e.g. `CONFIG_WOLFSSL=y` , see Espressif docs).

Algorithm Support & API Usage

X25519 (Curve25519 ECDH)

ESP32's crypto peripherals do *not* natively implement Curve25519, so X25519 runs in software. MbedTLS v4 exposes X25519 via its PSA interface. For example, one can use the PSA Crypto API:

```

psa_key_attributes_t attr = PSA_KEY_ATTRIBUTES_INIT;
psa_set_key_type(&attr, PSA_KEY_TYPE_ECC_KEY_PAIR(PSA_ECC_FAMILY_MONTGOMERY,
255));
psa_set_key_usage_flags(&attr, PSA_KEY_USAGE_DERIVE);
psa_set_key_algorithm(&attr, PSA_ALG_ECDH);
psa_set_key_bits(&attr, 255);
psa_key_id_t my_key;
psa_generate_key(&attr, &my_key); // local X25519 key
// Exchange public keys (X)
psa_export_public_key(my_key, pub_buf, pub_len, &olen);
// ... send pub_buf to peer, receive peer_pub ...
uint8_t shared[32];
size_t shared_len;
psa_key_derivation_operation_t op = PSA_KEY_DERIVATION_OPERATION_INIT;
psa_key_derivation_setup(&op, PSA_ALG_ECDH);
psa_set_key_derivation_key(&op, my_key);
psa_input_key(&op, PSA_KEY_DERIVATION_INPUT_PEER_KEY, peer_pub, peer_pub_len);
psa_key_derivation_output_bytes(&op, shared, sizeof(shared));
psa_key_derivation_abort(&op);

```

This uses `PSA_ALG_ECDH` with the 255-bit Montgomery curve (Curve25519). In fact, Mbed TLS's PSA implementation notes that "PSA_ALG_ECDH performs X25519 when used with this curve" [【65†L3478-L3482】](#) . The resulting 32-byte `shared` secret is identical to other X25519 implementations.

```

flowchart LR
    subgraph Alice
        A1[Generate private scalar a]
        A2[Compute A = a*G (public key)]
    end
    subgraph Bob
        B1[Generate private scalar b]
        B2[Compute B = b*G (public key)]
    end
    A1 --> A2 -->|Send A| Network((Network))
    B1 --> B2 -->|Send B| Network
    Network -->|Alice receives B| ComputeS1[Alice: S = a * B]
    Network -->|Bob receives A| ComputeS2[Bob: S = b * A]
    ComputeS1 --> Secret[Shared secret S]
    ComputeS2 --> Secret

```

Typical use: each side derives $S = a*B = b*A$. In benchmarks on an ESP32-S3, X25519 key generation takes ~14 ms and shared-secret computation ~15 ms [40†L210-L214] (software-only).

Interoperability: X25519 keys and secrets are standard RFC 7748. Keys generated on ESP32 (via Mbed TLS or WolfSSL) can interoperate with other libs (e.g. libsodium, OpenSSL).

Ed25519 (EdDSA signatures)

As of MbedTLS v4.x, Edwards-curve Ed25519 support is not implemented (the PSA spec indicates “Mbed TLS does not support Edwards curves yet”) [65†L3568-L3572]. Thus the built-in MbedTLS can’t do EdDSA natively. To use Ed25519 on ESP32, one can:

- Use PSA Crypto if future releases add Edwards-curve support.
- Use WolfSSL (wolfCrypt), which has had Ed25519 since ~2016 [66†L58-L62].
- Use libsodium (not hardware-accelerated on ESP32 [57†L0-L3]).
- Use NoxTLS, which explicitly lists Ed25519 support [22†L43-L51].

Example (WolfSSL/WolfCrypt API):

```

/* Generate Ed25519 key pair */
WC_RNG rng;
ed25519_key key;
wc_InitRng(&rng);
wc_ed25519_init(&key);
if (wc_ed25519_make_key(&rng, 32, &key) != 0) { /* handle error */ }
/* Sign a message */
uint8_t msg_hash[32] = { ... }; // SHA-256 of message (or use pure EdDSA
functions)
uint8_t sig[64];

```

```

wc_ed25519_sign_msg(msg_hash, sizeof(msg_hash), sig, &sig_len, &key);
/* Verify */
wc_ed25519_verify_msg(sig, sizeof(sig), msg_hash, sizeof(msg_hash), &key);

```

(Ed25519 in WolfSSL uses its own functions `wc_ed25519_*`.) In benchmarks, Ed25519 on ESP32-S3 takes ~26 ms to sign and ~24 ms to verify [\[40†L169-L177\]](#) .

Security: Ed25519 is fast and secure, but be sure to use a true RNG for key generation. Espressif’s hardware RNG can seed all library PRNGs (`esp_random()` or `bedtls_entropy`). No known hardware acceleration for Ed25519 exists, so software must run in constant-time mode (default in these libs).

HKDF (SHA-256)

HKDF (HMAC-based KDF) is supported in MbedTLS (`bedtls_hkdf()`) and in other libraries. In MbedTLS, call:

```

const mbedtls_md_info_t *md = mbedtls_md_info_from_type(MBEDTLS_MD_SHA256);
uint8_t prk[32], okm[32];
mbedtls_hkdf(md, salt, salt_len,
             ikm, ikm_len,
             info, info_len,
             okm, sizeof(okm));

```

Where `ikm` is input key material (e.g. shared ECDH secret), and `salt` / `info` are optional. For example, Espressif’s secure firmware encryption uses HKDF-SHA256: after computing an ECDH secret, it derives an AES-GCM key with HKDF (32-byte salt in header) [\[59†L144-L152\]](#) . The documentation notes: “KDF Salt (32 bytes): The salt used with HKDF (based on SHA256) to derive the AES-GCM encryption key from the ECDH shared secret” [\[59†L144-L152\]](#) .

Example:

```

uint8_t salt[32] = { /* from header or protocol */ };
uint8_t shared_secret[32] = { /* X25519 shared secret */ };
uint8_t aes_key[32];
mbedtls_hkdf(mbedtls_md_info_from_type(MBEDTLS_MD_SHA256),
             salt, sizeof(salt),
             shared_secret, sizeof(shared_secret),
             (const uint8_t*)"AES key", strlen("AES key"),
             aes_key, sizeof(aes_key));

```

Now `aes_key` can be used for AES-GCM.

AES-GCM (Authenticated Encryption)

AES-GCM is built into mbedTLS (`mbedtls_gcm_context`) and other libs. It uses hardware AES if enabled (on supported ESP32 chips). Basic usage in mbedTLS:

```
mbedtls_gcm_context ctx;
mbedtls_gcm_init(&ctx);
mbedtls_gcm_setkey(&ctx, MBEDTLS_CIPHER_ID_AES, key, 256); // key length in bits
// Encryption:
mbedtls_gcm_crypt_and_tag(&ctx, MBEDTLS_GCM_ENCRYPT, plaintext_len,
                          iv, iv_len, aad, aad_len,
                          plaintext, ciphertext, tag_len, tag);
mbedtls_gcm_free(&ctx);
```

This performs AES-GCM encrypt with associated data `aad` . For decryption, use `MBEDTLS_GCM_DECRYPT` and verify the tag. The Arduino example above showed the sequence of `mbedtls_gcm_setkey` , `mbedtls_gcm_starts` , `update` , `finish` , but newer mbedTLS provides the combined `crypt_and_tag` (and `auth_decrypt`) functions.

Example:

```
uint8_t key[32] = { ... };
uint8_t iv[12] = { ... };
uint8_t plaintext[] = "Hello, ESP32!";
uint8_t ciphertext[64], tag[16];
mbedtls_gcm_init(&ctx);
mbedtls_gcm_setkey(&ctx, MBEDTLS_CIPHER_ID_AES, key, 256);
mbedtls_gcm_crypt_and_tag(&ctx, MBEDTLS_GCM_ENCRYPT, strlen((char*)plaintext),
                          iv, sizeof(iv), NULL, 0,
                          plaintext, ciphertext, sizeof(tag), tag);
mbedtls_gcm_free(&ctx);
```

AES-GCM performance is fast with hardware AES: on ESP32-S3, AES-128-GCM encrypts ~1.35 MB/s [40†L100-L108] .

Projects/Repos Comparison

Library	License	X25519/ ECDH	Ed25519/ Sign	HKDF/SHA	AES-GCM	HW Accel (AES/SHA/ ECC)	Maturity Status
MbedTLS (Esp-IDF)	Apache-2.0 or GPL-2.0+ [62†L1354-L1362]	Yes (PSA) [65†L3478-L3482]	No (EdDSA not in mbedtls yet [65†L3568-L3572])	Yes [59†L144-L152]	Yes	Yes (AES/ SHA/ECC via HW) [9†L294-L302]	Official ESP-IDF very mature
NoxTLS (argenox)	GPL-2.0 or commercial [23†L1-L4]	Yes	Yes	Yes	Yes	Yes (configurable offload) [61†L169-L177]	Active, IDF compon
WolfSSL/ wolfCrypt	GPLv3 or commercial [56†L61-L69]	Yes	Yes	Yes	Yes	Yes (AES/ SHA/ECC)	Mature, Espress compon
libsodium (ESP port)	ISC (permissive)	Yes	Yes	Yes	Yes (via ChaCha20- Poly1305 or AES)	No (no ESP HW accel)	Mature, can compile ESP32
micro-ecc (uECC)	MIT (public domain variant)	Supports Curve25519? (no, only Weierstrass)	No	No	No	No	Simple library, small

Licenses: MbedTLS is dual Apache-2.0/GPL-2.0+ [\[62†L1354-L1362\]](#) . WolfSSL is GPLv3 (or commercial) [\[56†L61-L69\]](#) . NoxTLS is GPL-2.0 (or commercial) [\[23†L1-L4\]](#) . (Libsodium is ISC; uECC is MIT.)

Features: All listed libraries support HKDF and AES-GCM. X25519 support appears in all except basic uECC. Ed25519 is *not* in MbedTLS, but is in WolfSSL and NoxTLS (and libsodium). Hardware AES/SHA offload is used by MbedTLS, WolfSSL, NoxTLS via ESP32 crypto HALs; libsodium does not.

Security Considerations

- **Use HW RNG:** Always seed crypto PRNGs from the ESP32 hardware RNG (`esp_random()` or `mbedtls_hardware_poll()`). This ensures strong keys/nonces.
- **Key Protection:** On chips with the Digital Signature peripheral (ESP32-S2/S3/C3/C6), store RSA/EC private keys in the DS engine so they never leave hardware [\[59†L179-L187\]](#) . Enable `ESP32S2/`

Digital Signature Peripheral in menuconfig if available. (No analogous HSM for Ed25519 on ESP32.)

- **Disable Weak Algorithms:** In menuconfig → mbedTLS, disable MD5, RC4, and other obsolete ciphers (keep only AES, ChaCha20, SHA2, etc.). Always use TLS 1.2+ (CONFIG_MBEDTLS_TLS1_2 and TLS1.3) with AEAD ciphers.
- **Buffer Sizes:** Large TLS buffers consume RAM. If tight on RAM, reduce CONFIG_MBEDTLS_SSL_MAX_CONTENT_LEN below 16384 [47†L3287-L3294] , but ensure peers support short fragments.
- **Side-Channel Resistance:** By default mbedTLS/WolfSSL use constant-time implementations. Do not enable any “fast math” or optional CPU-accelerated ECDH that might skip constant-time measures.
- **Interoperability:** When exchanging keys or signatures with other systems, use standard formats. For example, export X25519 public keys as raw 32-byte values; use standard X.509 or raw signatures for Ed25519. MbedTLS’s PSA or PK abstractions typically handle format conversions.

Build Instructions & Example

1. **mbedTLS (ESP-IDF):** No extra steps needed – mbedTLS is built in. Simply include `<mbedtls/gcm.h>`, `<mbedtls/ecdh.h>`, etc. in your code. Configure needed algorithms in menuconfig (e.g. enable AES, GCM, ECP, ECDH). Then `idf.py build` normally. Use `mbedtls_*` APIs as shown above.

2. **NoxTLS:** In your project directory, run:

```
idf.py add-dependency "argenox/noxtls"
idf.py set-target esp32s3 # or esp32, esp32c3, etc.
```

Set the `sdkconfig.defaults` as in [61] to enable NoxTLS and hardware accel. Then `idf.py build flash monitor`. The library’s examples (`ports/esp-idf/examples/https_server`, etc.) demonstrate usage.

1. **WolfSSL:** Add as component (Espressif provides wolfSSL in component registry). In menuconfig, enable `CONFIG_WOLFSSL=y` (and optionally `CONFIG_WOLFSSL_CRYPTUART_HELPER`, etc.). Link against `wolfssl.lib`. Use WolfSSL API (or even esp-tls switch to wolfSSL) for X25519/Ed25519 as documented by WolfSSL.

References

- Espressif Mbed TLS programming guide (v6.0): shows config options and presets [9†L294-L302] [47†L3287-L3294] .
- Espressif mbedtls GitHub (fork info): “fork of Mbed TLS ... includes patches (hardware MPI, ECC)” [12†L202-L208] .

- Espressif *pre-encrypted image* component docs: example of ECDH+HKDF+AES-GCM, explaining HKDF salt usage [【59†L144-L152】](#) .
- Oryx-Embedded ESP32-S3 crypto benchmark: lists AES, SHA, ECC, X25519, Ed25519 timings [【40†L100-L108】](#) [【40†L169-L177】](#) [【40†L210-L214】](#) .
- NoxTLS component registry: features list including X25519/Ed25519/HKDF/AES-GCM [【22†L43-L51】](#) and setup instructions [【61†L138-L147】](#) .
- Mbed TLS v4 PSA reference: “PSA_ALG_ECDH performs X25519 when used with [Curve25519]” [【65†L3478-L3482】](#) ; “Mbed TLS does not support Edwards curves yet” (no Ed25519) [【65†L3568-L3572】](#) .
- WolfSSL blog: confirms support for Curve25519 and Ed25519 [【66†L58-L62】](#) .
- WolfSSL licensing page: “wolfSSL/wolfCrypt ... adheres to GPLv3” [【56†L61-L69】](#) .
- NoxTLS licensing (component): GPL-2.0-or-later or commercial [【23†L1-L4】](#) .
- MbedTLS license: dual Apache-2.0/GPL-2.0+ [【62†L1354-L1362】](#) .

Each citation above links to the relevant public source for verification. The information is up-to-date as of mid-2026.
